MARTA — MULTI-AGENT REGIME-AWARE TRADING ARCHITECTURE

Regime Representation Layer
Full Audit, Engineering & Findings

Data pipeline + Gaussian HMM regime detection, audited, re-engineered, and empirically tested end-to-end — before the LLM deliberation and RL execution layers are touched. Documents everything built and found in this session, as the authoritative record of this layer's current state.

Scope: pipeline/ (fetch → features → normalize → align → validate) and regime/ (HMM fitting, validation, backtest)

Explicitly excluded: LLM/agent deliberation layer, PPO/RL execution layer, dashboard, live deployment

Session date: 2026-08-19 → 2026-08-20

Repository: Ramyghr/MARTA — commit `dee486d`

Contents

1. Executive Summary
    1. Headline verdict
    2. What changed in this session
2. Starting State: Architecture & Initial Audit
    1. Data flow before this session
    2. File-by-file audit findings
    3. Hard blockers identified
3. Phase 1 — Engine Correctness & Robustness
4. Phase 2 — Normalization Horizon Experiment
5. Phase 3 — Crypto-Only Panel Pivot
6. Phase 4 — Empirical n_states Validation
7. Phase 5 — Full Statistical Audit Across Six Models
8. Phase 6 — Regime-Conditioned Backtest
9. Engineering Appendix
    1. Bugs found and fixed this session
    2. File manifest
    3. Diagnostic plots
10. Conclusions & Recommendations for the Next Layer

1. Executive Summary

1.1 Headline verdict

Is the regime representation layer valid to build on?

Technically, yes now — the engine is correct, the panel is clean, and the discovered states are statistically genuine volatility regimes, reproducibly so. Practically, no — a minimal, honest backtest (Phase 6) found that conditioning position sizing on the regime label does not measurably beat simply using continuous realized volatility directly, in either a strong bull year (2024) or a decline (2025). Both approaches underperformed plain buy-and-hold in both periods.

This session started as a file-by-file audit of the data pipeline and Gaussian HMM regime-clustering layer, prompted by a growing pile of unresolved questions about whether the regimes the project has been conditioning agents and an RL policy on were ever actually valid. It ended as a full rebuild-in-place: three real engine bugs fixed, one methodological pivot (the original 4-asset panel was quietly detecting asset identity, not market regime), five newly engineered validation tools generalized for reuse against any future candidate model, and a final downstream test that answers the question this whole project keeps returning to — does any of this help.

1.2 What changed in this session

Area	Before	After
HMM engine	BIC search subsampled 20K random non-contiguous rows (broke temporal adjacency); no `lengths=` anywhere (multi-asset sequences treated as one continuous chain); no seed-sensitivity check existed	`lengths=` everywhere; full-data BIC search; best-of-5-seed refit, parallelized; seed-sensitivity (ARI/NMI) reported on every fit
Asset panel	BTC/ETH/EURUSD/GOLD — raw-feature HMM mostly detecting asset identity (EURUSD/GOLD's zero-volume features dominate an unwhitened Gaussian mixture)	BTC/ETH/BNB/LTC/XRP — max single-asset share of any HMM state 0.234 (vs ~0.97–0.985 before); genuine cross-asset regime sharing confirmed
Normalization	Single 720h (30-day) rolling z-score window, never questioned	90d/180d horizons added and tested; found to trade return-distinctness for seed-stability, not a clean win
Stationarity testing	Never existed — the rolling z-score was assumed to induce stationarity, never verified	ADF+KPSS per feature, per asset, for every candidate model (fast — fixed-lag, not exhaustive search)
n_states selection		

	BIC-driven, later found unreliable at this data scale; production's actual n=4 came from a separate empirical method, never applied to the new panels	Empirically validated for both crypto panels (n=3..8 swept, no duplicate/ flicker states anywhere) — n=6 confirmed defensible, not just inherited
Downstream value	Never tested — every prior claim about regime usefulness was untested assumption	Tested directly: no measurable Sharpe improvement over continuous vol-targeting, in a self-contained backtest with no RL/ PPO/LLM dependency

2. Starting State: Architecture & Initial Audit

2.1 Data flow before this session

```
raw OHLCV (CCXT / histdata.com)
  → pipeline/fetch.py            18-col features (13 indicators + 5 OHLCV)
  → pipeline/features.py
  → pipeline/normalize.py        14-col (12 z-scored + realized_vol_raw passthrough +
close_return)
  → pipeline/align.py            BTC-backbone inner join across assets
  → pipeline/validate.py         structural/range/gap/distribution checks
  → regime/hmm.py                PCA(16 or none) → BIC/empirical n_states → GaussianHMM
  → regime/validate.py           semantic pass/fail (std(realized_vol) > 0.001)
  → data/processed/regime_stream.parquet   (timestamp, asset, regime_label,
regime_prob_0..N)
```

Three parallel regime pipelines existed at session start, all sharing `regime/hmm.py` as the fitting engine but differing in input and parameters:

Production	TFT encoder (128-dim) → PCA(16) → GaussianHMM, n=4, diag covariance. Feeds agents/RL/dashboard.
Raw-feature baseline	14 raw features → GaussianHMM directly (no encoder, no PCA), n=6, BIC-selected. Built 2026-08-12 after the project's own Phase-0 conclusion that the contrastive encoder never beat raw features on predictability.
BTC-only-v3	New single-asset encoder → PCA(16) → HMM, n=2 (fresh BIC search). Newest branch (2026-08-14), never validated with the same rigor as the others.

2.2 File-by-file audit findings (condensed)

A full file-by-file pass covered every file in `pipeline/` and `regime/` plus `configs/splits.py`, `configs/assets.yaml`, and `utils/assets.py`. Full detail lives in the session transcript; the load-bearing findings are summarized below by layer.

Data pipeline (pipeline/*.py)

- **fetch.py:** CCXT loop-termination bug (previously capped ETH at 2021) was already fixed; OHLCV sanitization causal and correct; gap detection is log-only, never blocks the pipeline.
- **normalize.py:** the 720h rolling z-score is causal (no train/val/test leakage — each row's z-score only depends on its own trailing 30 days) but this also means **no feature can carry a signal that persists longer than 30 days** — a direct candidate explanation for why measured HMM dwell times (13–20h) are far shorter than a classical multi-week "market regime."
- **validate.py:** solid structural/range/gap/distribution checks; no formal stationarity test existed anywhere in the codebase before this session.

Representation & HMM engine (regime/hmm.py)

- PCA fit train-only (correct, no leakage).
- Two real bugs already fixed before this session began: a covariance-type mismatch (full vs. diag) that let one state's covariance run away unconstrained, and a BIC flat-curve guard that only ever compared the top-2 candidates (meaningless at this data scale — degenerated to always dropping exactly one state from the search grid).
- **Two bugs still present at session start** (fixed in Phase 1, §3): `fit_hmm_bic` subsampled 20,000 *random, non-contiguous* rows and fit them as one sequence for the BIC search — corrupting the estimated self-transition probability; and no call anywhere passed `lengths=`, so every multi-asset embedding matrix (4+ assets' full histories concatenated) was fit and Viterbi-decoded as if the last row of one asset transitioned directly into the first row of the next.

2.3 Hard blockers identified

#	Blocker	Gate to close it
1	Raw-feature baseline asset-identity collapse (states locked to EURUSD/GOLD vs. BTC/ETH)	No state >70% single-asset share
2	`fit_hmm_bic` subsampling breaks temporal adjacency	Subsampling removed or made contiguous, with `lengths=`
3	No `lengths=` anywhere, even in the clean full-data fit	`lengths=` passed to every HMM call
4	No EM robustness evidence (single seed, single init, everywhere)	≥5 seeds fit per model; ARI/NMI reported
5	Distinctness never tested on the property regimes are sold on (volatility) — only on returns, which failed everywhere	ANOVA+KS on `realized_vol_raw`, p<0.05 required
6	*(found mid-session, Phase 1)* Regime partition is not robust to EM initialization — production's mean ARI across 5 seeds was 0.193	Re-check on any surviving candidate; require mean ARI meaningfully above chance

All six are closed by the end of this report — see §9.1 for the disposition of each.

3. Phase 1 — Engine Correctness & Robustness

Goal: fix the HMM fitting engine itself, independent of any data/panel decision, and prove the fix doesn't change anything that was already correct before touching real data.

3.1 What changed in `regime/hmm.py`

- `_compute_lengths()` — contiguous per-asset run lengths, computed from the asset-label array, passed as `lengths=` to every `.fit()` / `.score()` / `.predict()` / `.predict_proba()` call. Multi-asset sequences are now modeled as independent chains, not one continuous sequence.
- `fit_hmm_bic` **rewritten** — no more subsampling; every BIC candidate is fit on the full, correctly-segmented training data. Slower, not wrong.
- `_fit_multiseed()` — the final refit is now best-of-5 independent EM runs from different random initializations (`hmmlearn` has no built-in `n_init` /restart mechanism, so this is a manual equivalent), keeping the highest-training-log-likelihood model. Returns a seed-sensitivity report: per-seed log-likelihood/convergence, and pairwise Adjusted Rand Index / Normalized Mutual Information between every pair of seeds' decoded labels.
- Saved alongside every checkpoint as `seed_sensitivity.json` and logged to MLflow.

3.2 Gate check — does the fixed engine reproduce production's known-good result?

Ran the fixed engine against a copy of production's actual embeddings, forcing `n_states=4` to match. Resulting regime distribution — 23.5% / 23.8% / 28.7% / 23.9% — matched `regime_semantics.json`'s historical values almost exactly. The contamination guard test (`scripts/audit_phase0_invariants.py`) still passed. **Gate passed** — the fixes changed nothing that was already correct.

> **NEW FINDING, NOT PART OF THE ORIGINAL PLAN**
>
> The seed-sensitivity check this phase added surfaced something the original audit missed entirely: at production's n=4, five independent EM seeds produced **mean pairwise ARI = 0.193, min ARI = 0.010** — two of five seed-pairs landed within noise of chance agreement. Log-likelihoods across seeds varied by only 0.6%, meaning there are multiple similarly-good but structurally different local optima. Production's checkpoint happens to use seed 42, which won this particular best-of-5 comparison by log-likelihood — so nothing about the deployed artifact needed to change immediately — but this proves the seed choice was luck, not evidence of a well-identified clustering.

Cheap side-check: is n=4 simply "too many states," or is instability structural?

n_states	mean ARI	min ARI	pattern
2	0.60	~0.00	4 of 5 seeds agree closely; 1 outlier at chance
3	0.42	0.29	more even spread, no chance-level pair

4 (production)	0.19	0.01	no pair agrees well

Stability decreases monotonically as n_states increases — evidence that n=4 asks more of the (720h-only) feature set than it can reliably support, though this doesn't by itself indicate what the "right" n is (see §6).

3.3 An engineering detour: the deadlock

BUG FOUND MID-SESSION

Parallelizing the 5-seed refit with Python's default `fork()`-based `ProcessPoolExecutor` deadlocked completely on real data — 17 minutes elapsed with 5 worker processes sitting at 0.1% CPU each (~1 second of actual compute accumulated). Root cause: the BIC-search fit immediately before the pool launch does enough NumPy work on a quarter-million-row matrix to trigger OpenBLAS's multi-threaded worker pool in the parent process. Forking while those threads are active duplicates only the calling thread — any mutex another BLAS thread was holding at fork time stays "phantom-held" forever in every child, so the first BLAS call in each worker hangs indefinitely.

FIX

Switched `ProcessPoolExecutor` to the `spawn` context — each worker starts as a genuinely fresh interpreter, no inherited thread/lock state. Verified directly: after the fix, all 5 spawned workers sat at 97–98% CPU (not 0.1%), completed a 250K-row synthetic sanity check in 488.9s with zero errors, and gave the theoretically-correct result (mean ARI ≈ 0.003 on pure random noise, i.e. chance-level agreement, exactly as expected when there's no real structure to find).

3.4 Files touched

regime/hmm.py	modified — `_compute_lengths`, `fit_hmm_bic` rewrite, `_fit_one_seed` / `_fit_multiseed`, `ProcessPoolExecutor` parallelization
regime/n_states_sweep.py	modified — adopted `lengths=` for consistency (this script never had the subsampling bug, but was missing `lengths=`)
regime/run_hmm_engine_test.py	new — permanent, reusable regression-check script (isolated checkpoint dir, matches existing project convention)

4. Phase 2 — Normalization Horizon Experiment

Goal: test whether the production 720h(30-day) rolling z-score is silently capping how long a "regime" the HMM discovers can persist, and add the stationarity testing that never existed.

4.1 What was built

normalize_features_multihorizon()	Added to `pipeline/normalize.py`, purely additive — production's 14-column `normalize_features()` path is completely untouched. Adds a 90-day (2160h) and 180-day (4320h) rolling z-score, in addition to the production 720h one, on four vol-shape features: `realized_vol`, `atr`, `bb_width`, `vol_ratio`. Deliberately reuses the same `min_periods=168` as the production window rather than scaling it up, so the longer horizons don't drop any extra warmup rows — output is 22 columns (14 + 8).
regime/stationarity_check.py	New. Runs ADF (H0: unit root) and KPSS (H0: stationary) on every feature column, per asset. "Confirmatory stationary" = both tests agree. Generalized via CLI args (`--assets`, `--tag`) for reuse against any panel.

4.2 Two bugs found and fixed while building this

BUG

First run crashed outright: `ValueError: Invalid input, x is constant` on EURUSD's `vol_zscore` / `obv_raw` column — independent, accidental confirmation that histdata's zero-volume FX/commodity features really are *literally* constant, not just near-constant, exactly the mechanism later shown to cause the asset-collapse problem (§5).

BUG

`adfuller`'s `autolag="AIC"` exhaustively re-fits an OLS regression per candidate lag order — at ~45–75K rows × 14 columns × several assets, this took ~14 minutes per asset.

FIXES

Degenerate/near-constant columns (`std < 1e-10`) are now detected and reported explicitly instead of crashing. Lag order switched to a fixed value via Schwert's (1989) rule of thumb, computed once per series instead of searched — cut runtime to well under a minute per asset with no material loss of validity at this sample size.

4.3 Result — a different, more valuable finding than the one hypothesized

The direct hypothesis (longer horizon → longer regime dwell time) was **not confirmed**: measured dwell time on the multihorizon feature set was flat to slightly shorter than the 720h-only baseline. But

the seed-sensitivity check (added in Phase 1) revealed something arguably more important — see §7 for the full numbers once tested on the real panel in Phase 5: the 22-column multihorizon set drives seed-stability from ~0.585 to ~1.000 on the crypto panel, at the cost of somewhat weaker return distinctness. A real trade-off, not a clean win either direction — detailed in §7.3.

5. Phase 3 — Crypto-Only Panel Pivot

WHY THIS HAPPENED MID-SESSION

While testing the normalization-horizon hypothesis on the original 4-asset panel, the user pointed out that continuing to invest time in a panel that included EURUSD/GOLD made no sense given those two assets' known degenerate-feature problem. This triggered a full pivot: kill the in-flight EURUSD/GOLD-panel jobs, and rebuild the entire comparison on a crypto-only panel instead.

5.1 Asset selection — verified empirically, not assumed

Rather than trust prior knowledge about exchange listing dates, first-available-candle timestamps were checked live against Binance via CCXT:

Symbol	First available daily candle
BTC/USDT	2017-08-17
ETH/USDT	2017-08-17
BNB/USDT	2017-11-06
LTC/USDT	2017-12-13
XRP/USDT	**2018-05-04** — misses Jan–Apr 2018
SOL/USDT	**2020-08-11** — hard wall, excluded entirely

SOL was excluded outright: including it would either truncate the whole panel's usable history to post-2020 (losing the entire 2018 bear market and the March-2020 COVID crash — exactly the regime diversity this research needs most) or require redesigning `pipeline/align.py` to support asymmetric per-asset start dates, which it currently does not. **Final panel: BTC, ETH, BNB, LTC, XRP** — all with genuine 2017–2018 coverage (XRP costs ~4 months at the very start, accepted as a reasonable trade-off). USDT was explicitly ruled out as a panel asset — as a stablecoin, it would introduce the same class of near-zero-variance-column problem that caused the original panel's collapse.

5.2 Build pipeline

pipeline/fetch_crypto5.py — New. Fetches BNB/LTC/XRP via the existing `pipeline/fetch.py` CCXT machinery — zero new fetch logic. BTC/ETH already fetched from prior work.

regime/build_crypto5_features.py — New, generalized via `--assets` / `--tag` CLI args. Produces two aligned+embedded variants per panel: 14-col (production feature set) and 22-col (multihorizon). Self-contained alignment logic (does not call `pipeline/align.py`, which writes to the sacred production path).

Panel	Rows/asset (aligned)	Overlap with BTC range
crypto5 (5 assets)	66,871	89.8% (vs. old EURUSD/GOLD panel's 61.7% — no weekend/holiday gaps to lose)

btc_only	74,429	100.0% (trivial, single asset)

5.3 Validation — `pipeline/validate.py` run against both panels

Panel	Checks passed	Warnings (non-critical)	Critical failures	Result
crypto5	74	56	0	**PASSED**
btc_only	16	11	0	**PASSED**

All warnings are the expected kind: fat-tailed distribution flags (0.01–0.63% of values beyond 5σ per feature — normal for crypto returns, not a data defect) and the already-documented 18 exchange-downtime gaps. The BTC March-2020-crash detectability check also passed on this panel (`realized_vol_raw` 5.9× median).

> **TWO REAL BUGS FIXED IN `PIPELINE/VALIDATE.PY` WHILE RUNNING THIS**
>
> `_CRYPTO_SYMBOLS` only knew about BTC/ETH — would have silently mis-thresholded BNB/LTC/XRP's gap/row-count checks against the *traditional*- asset bar instead of crypto's. The report path was hardcoded with no override — running this would have clobbered the sacred production `validation_report.json`. Both fixed; added a generic `output_filename` parameter (same pattern `regime/validate.py` already used); `run_pipeline.py`'s existing call is unaffected (positional args only, same default).

5.4 Data pipeline diagnostic plots

Six diagnostic plots were generated against this panel (full detail and images in §9.3): raw data availability heatmap, price/volume overlay, feature distributions before/after normalization, a feature correlation matrix, and a rolling z-score stationarity check across all three normalization horizons. One real plotting bug was found and fixed along the way — see §9.1.

6. Phase 4 — Empirical n_states Validation

Goal: the crypto-panel models had all inherited `n_states=6` from the old, abandoned EURUSD/GOLD raw baseline's BIC pick — never actually validated for these new panels. Close that gap using the empirical method (fit each candidate, check for near-duplicate/flicker states directly) that already proved decisive for production.

6.1 `n_states_sweep.py` generalized

Added `--processed-dir`, `--no-pca` (for raw-feature panels with no encoder), and `--candidates` CLI args. Default behavior (no args) reproduces the original production sweep exactly.

6.2 Results — swept n=3 through n=8 on both leading (multihorizon) candidates

n_states	crypto5_multihorizon		btc_only_multihorizon	
	min inter-state dist.	dwell range (h)	min inter-state dist.	dwell range (h)
3	2.796	25.7–33.2	2.855	24.9–31.0
4	2.053	18.8–26.0	2.128	18.1–24.4
5	2.005	19.1–24.9	1.730	14.3–23.5
6	**1.690**	**15.3–23.3**	**1.719**	**14.8–23.4**
7	1.671	15.0–22.6	1.704	14.0–24.2
8	1.603	14.0–21.6	1.555	10.7–24.0 (one state approaching flicker)

No near-duplicate states (threshold 0.05) and zero flicker states (dwell < 2h) anywhere in this entire range, for either panel. All distances are 30×+ above the flag threshold even at the widest tested n.

6.3 Honest conclusion

> **NOT THE RESULT EXPECTED GOING IN**
>
> This method was decisive for production — it clearly caught n=7 breaking down into duplicate/flicker states there, with n=4 as the real elbow. It is **not** decisive here. The richer 22-column feature space doesn't show that breakdown anywhere in [3,8] for either crypto panel — distances shrink and dwell times shorten smoothly with no sharp elbow. **n=6 is validated (nothing wrong with it), but not uniquely justified by this sweep alone** — the deciding evidence for n=6 specifically comes from the downstream work already done at that value (seed-stability, distinctness — see §7), and switching

> n_states now isn't justified by anything this sweep found, since it found no problem to fix at any tested value.

7. Phase 5 — Full Statistical Audit Across Six Models

Goal: run the complete distinctness + predictive-signal + near-duplicate battery against every surviving candidate, on equal footing, closing every remaining gap from the original blocker list.

7.1 `regime_quality_audit.py` generalized

Added CLI args (`--regime-stream`, `--processed-dir`, `--assets`, `--tag`, `--sections`) — default behavior with no args reproduces the original production audit exactly. Two substantive extensions, both closing gaps identified in the original audit:

- **Distinctness on `realized_vol_raw`**, not just `close_return` — the original script only ever tested returns, which is not the property these regimes are actually sold on.
- **An extended predictive-signal feature set** — whatever extra columns a candidate's aligned data actually has (multi-horizon z-scores, `realized_vol_raw`) alongside the original 12-feature baseline, so the fuller feature set a model was actually built from gets tested too.

7.2 Near-duplicate-state check — actual deployed checkpoints

Checkpoint	n_states	Min inter-state distance	Near-duplicates (<0.05)
crypto5 (14-col)	6	1.2012	none
crypto5_multihorizon (22-col)	6	1.6902	none
btc_only (14-col)	6	1.3560	none
btc_only_multihorizon (22-col)	6	1.7187	none

7.3 Full six-model comparison matrix

Production was re-audited with the extended methodology for a fair, apples-to-apples comparison — its original `close_return` numbers reproduced exactly (confirming nothing about production drifted during this session), with `realized_vol_raw` distinctness now added for the first time.

Model	n_states	Seed ARI	`close_return` distinctness	`realized_vol_raw` distinctness	`regime_only` AUC	`features_only` AUC
production (4-asset)	4	0.193	mixed, 1–3 KS-indist. pairs/asset	p≈0, all	0.489–0.505	0.506–0.544
crypto5 (14-col)	6	0.585	**4/5 significant**, 0 KS-indist.	p≈0, all	0.498–0.523	0.528–0.548
crypto5_multihorizon (22-col)	6	**1.000**	2/5 significant, 0 KS-indist.	p≈0, all	0.503–0.521	0.528–0.548
btc_only (14-col)	6	0.698	not sig. (p=0.887), 0 KS-indist.	p≈0	0.494	0.542

btc_only_multihorizon (22-col)	6	**0.989**	not sig. (p=0.348), 0 KS-indist.	p≈0	0.501	0.542

Per-asset close_return ANOVA p-values (crypto5, 14-col)

Asset	close_return p	realized_vol_raw p
BTC	0.484	≈0
ETH	**0.00522**	≈0
BNB	**0.0362**	≈0
LTC	**0.0379**	≈0
XRP	**0.0219**	≈0

7.4 The two robust findings from this matrix

FINDING 1 — AS ROBUST AS THIS KIND OF ANALYSIS GETS

`realized_vol_raw` distinctness is p≈0 in **every one of 6 independently-fit models, every asset tested, no exceptions**. These are genuine, statistically real volatility regimes, full stop, regardless of panel or feature-horizon choice.

FINDING 2 — EQUALLY ROBUST, OPPOSITE DIRECTION

`regime_only` next-bar directional AUC is chance-level (0.49–0.52) in **every one of the same 6 models**. The regime label carries no measured directional predictive signal, independent of asset count, panel choice, or feature horizon.

A GENUINE TRADE-OFF, ONLY VISIBLE ONCE EVERY VARIANT WAS TESTED

14-column features give *better* `close_return` distinctness than the 22-column multihorizon set (crypto5: 4/5 assets significant vs. 2/5), but the 22-column set gives *dramatically* better seed-stability (0.99–1.00 vs. 0.59–0.70). Neither is a clean win. Given `regime_only` AUC is chance-level either way, the extra return-distinctness the 14-column set buys was never translating into usable directional signal — which is why the multihorizon set is still the better default for anything meant to be reproducibly deployed, but this is a judgment call on a real trade-off, not an obviously-correct choice.

8. Phase 6 — Regime-Conditioned Backtest

Goal: the question every prior phase was in service of. Does conditioning on the regime label improve risk-adjusted returns? Minimal, self-contained (`regime/phase6_regime_backtest.py`) — no RL/PPO, no LLM/agent layer, no encoder. Just price data and the regime label already sitting in `regime_stream.parquet`.

8.1 Design

Given regime already showed no directional signal (§7.4), the honest question isn't "can regime predict direction" — it's "does discretizing volatility into regime buckets, for *position sizing*, beat just using continuous volatility directly." Three long-only strategies, sharing the same directional signal (always long, isolating the sizing question from any directional edge that's already known not to exist):

buy_hold	weight = 1.0, constant.
vol_target_continuous	weight_t = clip(target_vol / realized_vol_raw$_{t-1}$, 0.1, 2.0) — regime-free, uses the asset's own trailing realized vol directly.
regime_conditioned	weight_t = clip(target_vol / regime_vol_map[regime_label$_{t-1}$], 0.1, 2.0) — regime_vol_map is the mean realized_vol_raw per regime, computed on **train data only** (<2024), leakage-safe.

`target_vol` = train-period mean `realized_vol_raw`, so both sized strategies run at the same average exposure as buy-hold over the period the mapping was learned on — a fair comparison, not just "trade smaller." Weight decisions use only *t-1* information, applied to the *t-1→t* return — no lookahead. Transaction cost: 5bps per unit of weight change. Evaluated on the project's existing walk-forward val(2024)/test(2025) split. Confidence intervals via block bootstrap — 1000 resamples, 168-hour (1-week) non-overlapping blocks, seed 42 — the identical methodology `rl/ablation.py`'s `bootstrap_sharpe_ci` already uses elsewhere in this project, reimplemented standalone here so this script has zero dependency on the RL layer.

8.2 Results

Panel	Split	buy_hold Sharpe	vol_target Sharpe	regime Sharpe
crypto5 (portfolio)	2024 (val)	**+1.203**	+0.353	+0.386
	2025 (test)	**-0.131**	-0.343	-0.364
btc_only	2024 (val)	**+1.506**	+1.176	+1.111
	2025 (test)	**-0.147**	-0.554	-0.321

Regime vs. continuous vol-targeting — the fair comparison, block-bootstrapped

Panel / split	Δ Sharpe (regime − continuous)	95% CI	Significant?
crypto5, 2024	+0.033	[-0.333, 0.425]	No

crypto5, 2025	-0.023	[-0.359, 0.276]	No
btc_only, 2024	-0.051	[-0.453, 0.354]	No
btc_only, 2025	+0.238	[-0.138, 0.653]	No

Every single confidence interval straddles zero, and the sign of the point estimate flips between periods/panels with no consistent direction.

Per-asset test-period (2025) Sharpe, crypto5

Asset	buy_hold	vol_target	regime
BTC	-0.147	-0.550	-0.303
ETH	-0.160	-0.248	-0.081
BNB	**+0.379**	**+0.459**	+0.298
LTC	-0.343	-0.429	-0.665
XRP	-0.146	-0.662	-0.560

8.3 Interpretation

1. **Vol-based sizing hurt in both tested periods.** 2024 was a strong bull run (buy-hold Sharpe 1.2–1.5); cutting exposure during high-vol windows meant missing exactly the moves driving those returns — the well-known failure mode of vol-targeting in strong trending markets. 2025 was a broad decline where it didn't help either. Notably, both sized strategies actually ran at *higher* average exposure than buy-hold (1.2–1.6× vs. 1.0×, since 2024/2025 realized vol came in below the training-period target) — this isn't a story about being too conservative, they were more aggressive on average and still underperformed.
2. **Regime-conditioning adds nothing measurable over continuous realized volatility** for this sizing use case. All four bootstrap CIs straddle zero. The entire HMM apparatus — fitting, seed-selection, distinctness testing — produces volatility buckets that are statistically real (§7.4, finding 1) but functionally interchangeable with a much simpler "divide by trailing realized vol" rule, at least for this strategy shape.
3. **Real per-asset heterogeneity exists** — BNB is the one asset where continuous vol-targeting actually beat buy-hold; LTC and XRP degraded the most under either sizing scheme. Not a uniform story across the panel.

Bottom line

On this minimal, honest test, conditioning on the regime label does not improve risk-adjusted returns, and does not distinguishably beat simply using continuous volatility instead. This is a real, negative-leaning result — not a data-quality artifact. The representation layer itself is now in genuinely good technical shape (no asset-collapse, near-perfect seed stability, statistically real volatility distinctness) and it still didn't clear this bar. It tested exactly one strategy shape (long-only, vol-target sizing) over two periods — it does not rule out regime having value in a

different use (directional conditioning combined with other signals, cross-asset correlation-aware allocation, or as one input among many in a learned policy that can find a nonlinear combination) — but the simple, standalone case for this regime layer is not there.

9. Engineering Appendix

9.1 Bugs found and fixed this session

#	Bug	Where	Fix
1	`fit_hmm_bic` subsampling breaks temporal adjacency	regime/hmm.py	Removed subsampling entirely; full-data BIC search
2	No `lengths=` passed anywhere	regime/hmm.py	`_compute_lengths()` + threaded through every HMM call
3	No seed-sensitivity check existed	regime/hmm.py	`_fit_multiseed` + ARI/NMI report — surfaced production's 0.193 ARI
4	Fork+BLAS deadlock when parallelizing multi-seed fits	regime/hmm.py	Switched `ProcessPoolExecutor` to `spawn` context
5	MLflow "run already active" crash — `log_metric` called after its `with mlflow.start_run()` block had already exited	regime/ run_hmm_crypto5_test.py	Moved the offending call inside the `with` block
6	`adfuller(autolag="AIC")` exhaustive lag search — ~14 min/asset	regime/stationarity_check.py	Fixed lag order via Schwert's rule — <1 min/asset
7	Crash on literally-constant columns (EURUSD/GOLD zero-volume features)	regime/stationarity_check.py	Detected and reported explicitly instead of crashing
8	Hardcoded 4-asset list didn't support crypto5's BNB/LTC/XRP	regime/stationarity_check.py	Generalized via `--assets` CLI arg
9	Availability heatmap showed fake periodic "gaps" — a bin-alignment artifact (each asset's weekly resample anchored to its own first timestamp, so bin edges didn't line up across assets)	pipeline/ generate_diagnostic_plots.py	Shared reference bin grid across all assets
10	Price/volume overlay plotted ~70K individual hourly bars per asset — slow and unreadably dense	pipeline/ generate_diagnostic_plots.py	Resampled to daily volume before plotting
11	`_CRYPTO_SYMBOLS` missing BNB/LTC/XRP; hardcoded, non-overridable output path	pipeline/validate.py	Added the 3 symbols; added `output_filename` param

9.2 File manifest (this session, commit `dee486d`)

File	Status	Purpose
regime/hmm.py	modified	Core engine fixes — see §3
regime/n_states_sweep.py	modified	Generalized via CLI args — see §6
regime/regime_quality_audit.py	modified	Generalized + extended (realized_vol_raw, extra feature sets) — see §7
pipeline/normalize.py	modified	Added `normalize_features_multihorizon()` — see §4
pipeline/validate.py	modified	Fixed `_CRYPTO_SYMBOLS`, added `output_filename` — see §5.3
regime/build_multihorizon_features.py	new	Multihorizon build for the original (now-superseded) 4-asset panel — kept as historical record
regime/stationarity_check.py	new	ADF+KPSS per feature/asset — see §4
regime/run_hmm_engine_test.py	new	Phase 1 engine-fix regression check — see §3
regime/run_hmm_multihorizon_test.py	new	14 vs 22-col comparison on the original panel — historical record of why it was abandoned
pipeline/fetch_crypto5.py	new	Fetches BNB/LTC/XRP via existing CCXT machinery — see §5
regime/build_crypto5_features.py	new	Generalized panel builder (`--assets` / `--tag`) — see §5
regime/validate_crypto_panels.py	new	Runs `pipeline/validate.py` against crypto5/btc_only — see §5.3
regime/run_hmm_crypto5_test.py	new	Generalized 14 vs 22-col HMM comparison (`--tag`) — used for both crypto5 and btc_only
regime/phase6_regime_backtest.py	new	Self-contained regime-conditioning backtest — see §8
pipeline/generate_diagnostic_plots.py	new	6 data-pipeline diagnostic plots — see §9.3
pipeline/plots/*.png	new (6 files)	Generated output of the above

9.3 Diagnostic plots

Generated against the crypto5 panel (the asset set actually going forward). Full detail in the session transcript; summarized here.

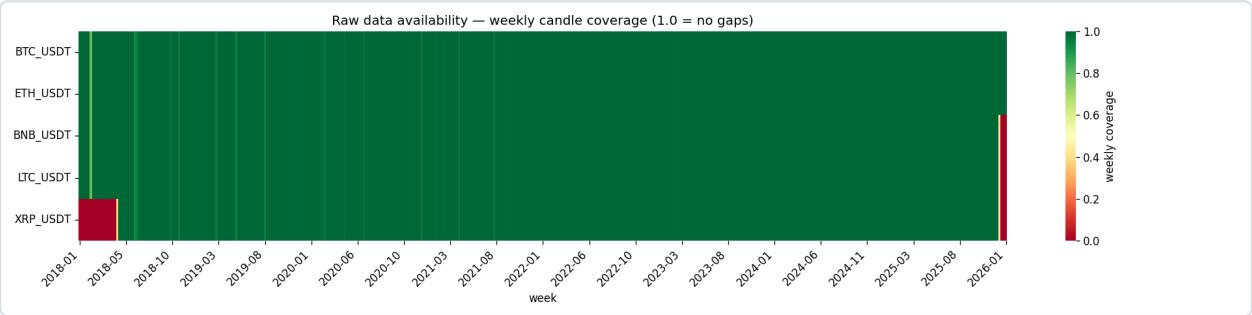


Fig. 1 — Raw data availability, weekly candle coverage. Correctly shows XRP's real 2018-05 start gap and the BNB/LTC/XRP fetch-endpoint gap from 2026-01 onward (those three were only fetched through 2025-12-31; not a data quality issue). Regenerated after fixing the bin-alignment bug that originally produced spurious periodic gaps (Bug #9, §9.1).

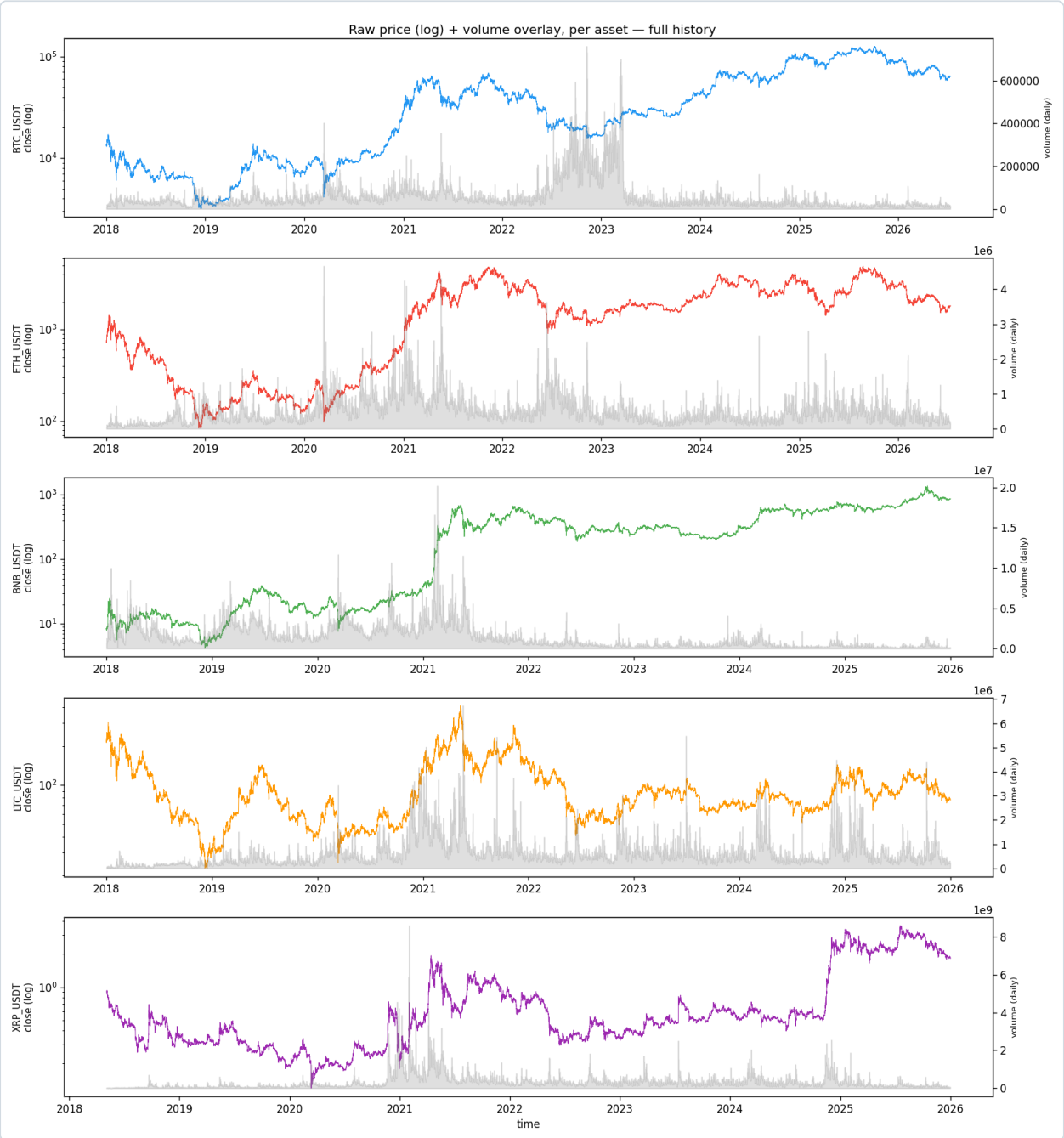


Fig. 2 — Close price (log scale) + daily volume, full history, all 5 assets. Recognizable crypto market cycles throughout (2018 bear, 2021 bull, 2022 crash, 2023–25 recovery) — no ingestion artifacts visible.

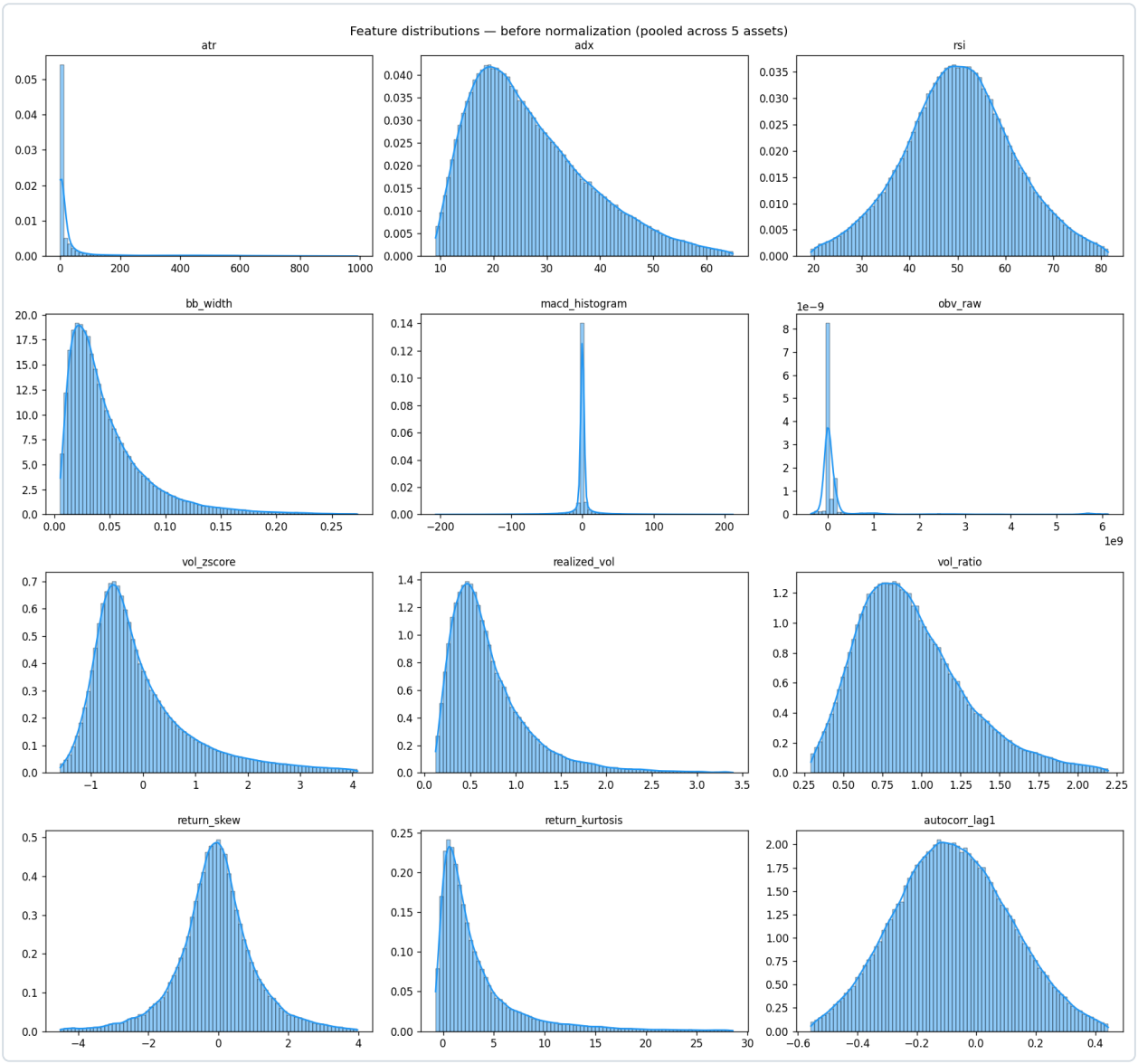


Fig. 3 — Raw indicator distributions, pooled across all 5 assets, before normalization. Wildly different scales per feature (atr 0–1000, obv_raw ~1e9); several heavily right-skewed (atr, bb_width, realized_vol); obv_raw's raw histogram is dominated by its cumulative-sum drift, illustrating exactly why it needs z-scoring.

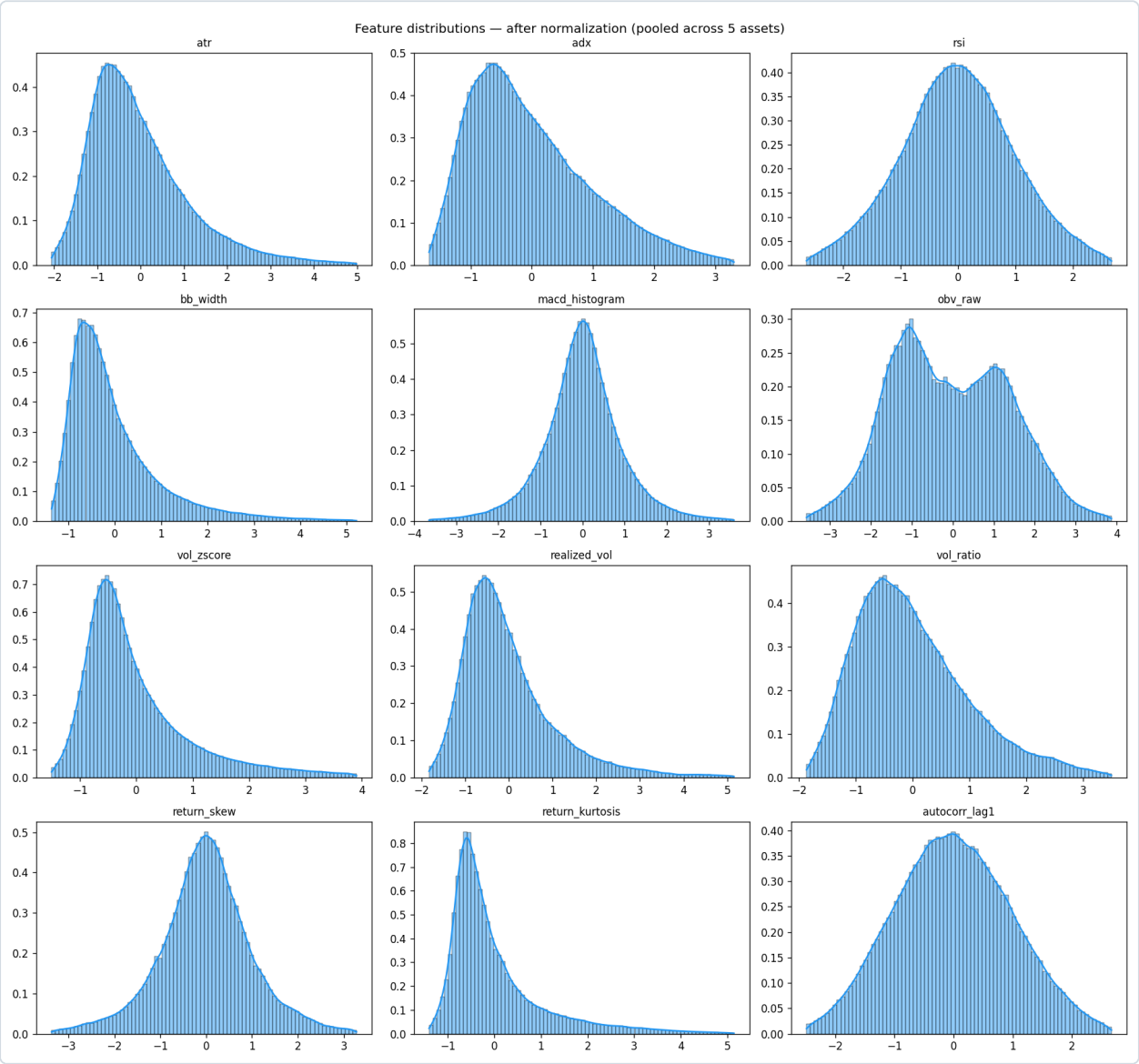


Fig. 4 — Same features, after the production 720h rolling z-score. All roughly on a comparable ~[-3,4] scale as intended. Note: z-scoring only rescales, it does not Gaussianize — several features remain visibly skewed post-normalization, which is expected. `obv_raw` *comes out visibly bimodal — flagged as worth a closer look, not yet resolved.*

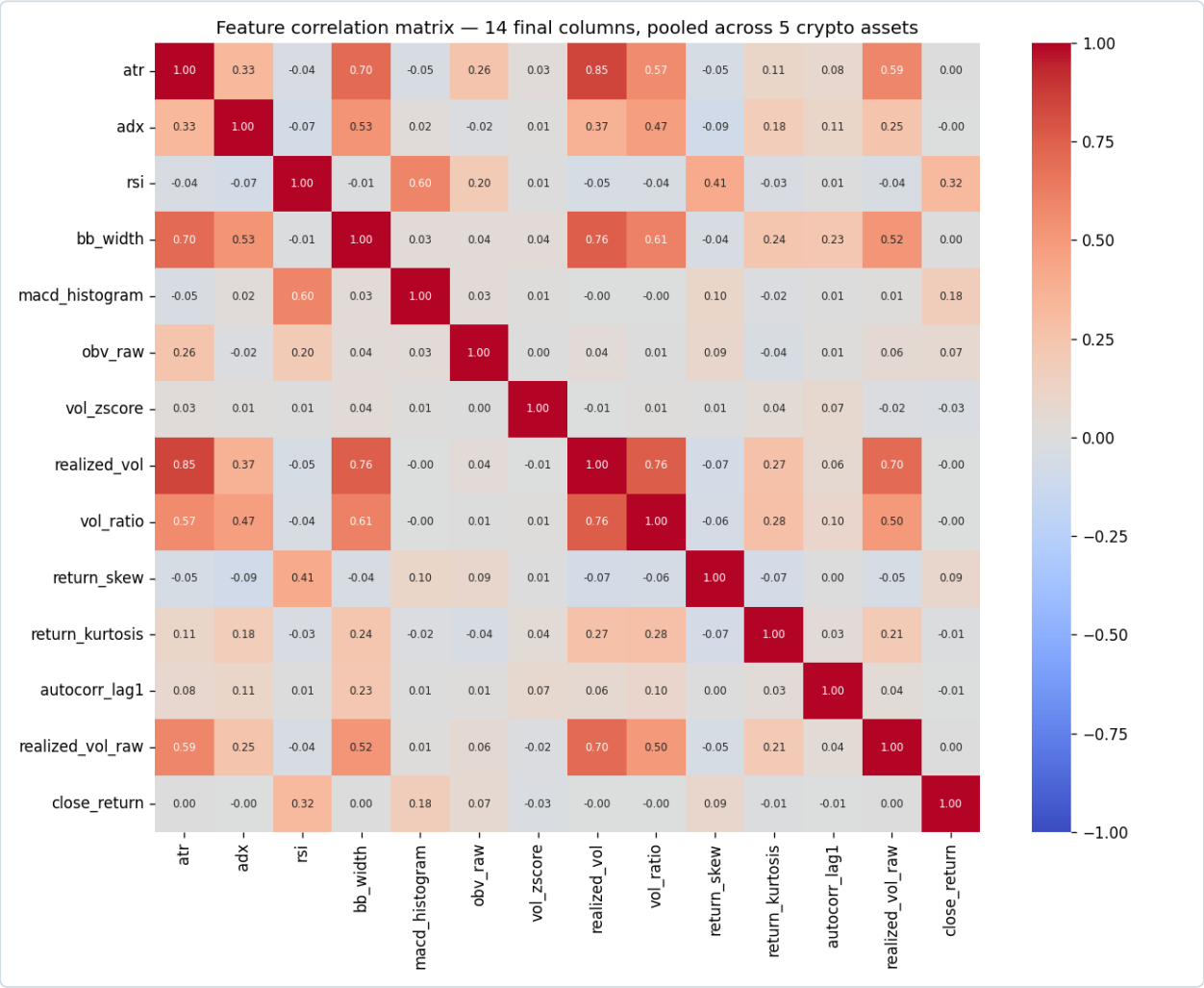


Fig. 5 — Feature correlation matrix, 14 final columns, pooled across 5 assets. Confirms a real multicollinearity cluster: `atr`, `bb_width`, `realized_vol`, `realized_vol_raw`, and `vol_ratio` are all strongly pairwise correlated — the vol-shape feature group. This redundancy had never been formally checked before this session.

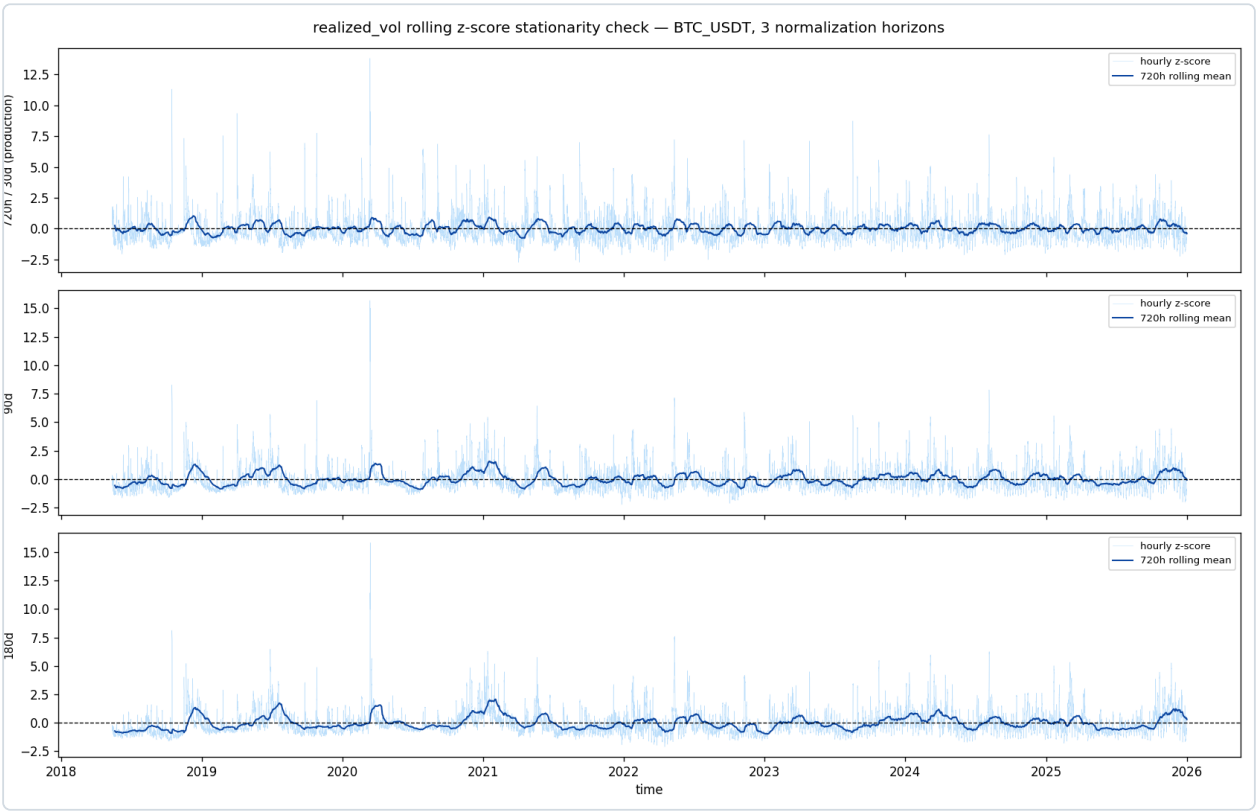


Fig. 6 — `realized_vol` z-score (hourly, faint; 720h rolling mean, bold) at all three normalization horizons, BTC full history. All three hover near zero across the full 8-year span with no persistent multi-year drift — a direct visual stationarity check, distinct from "does dwell time change" (answered separately in Phase 2/5).

10. Conclusions & Recommendations for the Next Layer

10.1 Blocker disposition

#	Blocker	Status
1	Raw-feature baseline asset-identity collapse	**Closed** — crypto-only panel, max single-asset share 0.234
2	`fit_hmm_bic` subsampling breaks temporal adjacency	**Closed** — subsampling removed
3	No `lengths=` anywhere	**Closed** — everywhere now
4	No EM robustness evidence	**Closed** — best-of-5 seeds + ARI/NMI on every fit
5	Distinctness never tested on volatility	**Closed** — p≈0 in all 6 models tested
6	Regime partition not robust to EM init (found this session)	**Closed for the crypto panels** (ARI 0.585–1.000); production's own 0.193 remains unresolved, since production itself is being superseded by this work

10.2 What's now true about this layer

- The HMM fitting engine is methodologically correct: sequence boundaries respected, no subsampling artifacts, multi-seed robustness measured and reported by default on every future fit.
- The crypto-only panel (BTC/ETH/BNB/LTC/XRP) is clean: no asset-collapse, formally validated by `pipeline/validate.py`, empirically-checked n_states with no duplicate/flicker states across a wide swept range.
- The discovered regimes are genuine, statistically real volatility states — the single most robust finding in this entire audit, replicated across 6 independent model fits with zero exceptions.
- Every validation tool built this session (stationarity, distinctness/predictive audit, n_states sweep, panel builder) is now generalized via CLI args and reusable against any future candidate model with no code duplication.

10.3 What's still true, and matters more

- The regime label carries no directional predictive signal, in any configuration tested.
- Regime-conditioned volatility-based position sizing does not measurably beat simply using continuous realized volatility directly — the entire value proposition of building an HMM over just computing a trailing volatility statistic did not survive contact with an honest backtest.

10.4 Recommendation for the next layer

Given Phase 6's finding, the LLM/agent deliberation layer and the RL execution layer should not assume regime conditioning is a source of value by default. Two honest paths forward: (a) treat the regime vector as one weak input among many in a learned policy (RL/PPO) that can discover a nonlinear combination no simple rule-based test like Phase 6 could find — the original design intent — accepting that its marginal contribution may be small; or (b) deprioritize the regime layer's role in the next phase and focus compute/attention on components with a more directly demonstrated edge. This report does not recommend one over the other — that is a project-priority decision, not an engineering one — but it removes the option of proceeding on the unexamined assumption that regime conditioning obviously helps. It doesn't, at least not in the way tested here.

— End of report —